Karadeniz Teknik Üniversitesi
Fen Fakültesi – Bilgisayar Bilimleri Bölümü

# Hızlı Başvuru Kağıdı #2: İteratörler ve STL Algoritmaları

Yazılım Geliştirme II – Hafta 2

## İteratör Kategorileri

**Hiyerarşi (Zayıf → Güçlü)**

| Kategori | Özellik | Örnek |
|---|---|---|
| Input | Oku, tek geçiş | `istream_iterator` |
| Output | Yaz, tek geçiş | `ostream_iterator` |
| Forward | Oku/yaz, çok geçiş | `forward_list` |
| Bidirect. | + geri hareket | `list`, `map`, `set` |
| Random Acc. | + aritmetik | `vector`, `deque` |

**begin / end Ailesi**

```
v.begin()  / v.end()      // normal
v.cbegin() / v.cend()     // const
v.rbegin() / v.rend()     // ters
std::begin(arr)            // C dizileri icin
std::end(arr)
```

**İteratör Adaptörleri**

```
#include <iterator>
std::back_inserter(v)    // push_back
std::front_inserter(d)   // push_front
std::inserter(v, pos)    // insert(pos,...)
std::ostream_iterator<int>(cout, " ");
```

**İteratör Geçersizleşmesi**

`vector`: ekleme/silme sonrası geçersiz olabilir. `map/set`: silinen iteratör hariç diğerleri geçerli kalır. Kural: Döngüde silme yaparken `it = v.erase(it)` kullan!

## Temel Algoritmalar

**Genel Kalıp**

`#include <algorithm>`
Algoritmalar iteratör aralıkları üzerinde çalışır: `[begin, end)`

**Sıralama**

```
std::sort(v.begin(), v.end());        // kucuk->buyuk
std::sort(v.begin(), v.end(),         // ozel kriter
    [](int a, int b){ return a > b; });
std::stable_sort(v.begin(), v.end()); // goreli sira korunur
std::partial_sort(v.begin(),
    v.begin()+3, v.end());  // ilk 3 sirali
```

**Arama**

```
auto it = std::find(b, e, 42);        // deger ara
auto it = std::find_if(b, e,          // kosul ara
    [](int x){ return x > 50; });
bool ok = std::binary_search(b,e,42);// sirali!
auto [lo,hi] = std::equal_range(b,e,42);
```

**Sayma ve Dolaşma**

```
int n = std::count(b, e, 42);
int n = std::count_if(b, e, pred);
std::for_each(b, e,
    [](int x){ std::cout << x << " "; });
```

## Kopyalama ve Dönüştürme

**copy / transform**

```
// Kopyalama (hedefte yer olmali!)
std::copy(b, e, std::back_inserter(v2));
std::copy_if(b, e, std::back_inserter(v2),
    [](int x){ return x > 0; });

// Donusturme
std::transform(b, e, b,              // yerinde
    [](int x){ return x * 2; });
std::transform(b, e,                 // yeni hedef
    std::back_inserter(sonuc),
    [](int x){ return x * x; });
```

## Silme: Remove-Erase

**Klasik Deyim (C++98)**

```
// remove sadece sona tasir, silmez!
auto it = std::remove(b, e, 42);
v.erase(it, v.end());  // gercek silme

// Kosullu silme
auto it = std::remove_if(b, e, pred);
v.erase(it, v.end());
```

**C++20 Kısa Yol**

```
std::erase(v, 42);         // tek satirda sil
std::erase_if(v,           // kosullu sil
    [](int x){ return x < 0; });
// Tum kapsayicilar icin calisiyor!
```

**std::remove Tuzağı**

`remove` kapsayıcının `size()`'ını **değiştirmez**. Sadece elemanları sona taşır ve yeni mantıksal sonu döndürür. Mutlaka `erase` ile birlikte kullan (veya C++20 `std::erase`).

## İleri Algoritmalar

**accumulate / reduce**

```
#include <numeric>
int top = std::accumulate(b, e, 0);   // toplam
int crp = std::accumulate(b, e, 1,
    std::multiplies<int>());          // carpim
// C++17: paralel versiyonu
auto r = std::reduce(b, e, 0);
```

**partition / nth_element**

```
// Kosulu saglayanlari basa tasi
auto p = std::partition(b, e,
    [](int x){ return x % 2 == 0; });
// p: ikinci grubun basi

// n. elemani dogru konuma yerlestir
std::nth_element(b, b+2, e);
// b[2] sirali olsaydi o konumdaki deger
```

## Küme İşlemleri (Sıralı Aralık)

**set_union / set_intersection**

```
std::vector<int> a={1,3,5}, b={2,3,4};
std::vector<int> sonuc;
// Birlesim: {1,2,3,4,5}
std::set_union(a.begin(), a.end(),
    b.begin(), b.end(),
    std::back_inserter(sonuc));
// Kesisim: {3}
std::set_intersection(a.begin(), a.end(),
    b.begin(), b.end(),
    std::back_inserter(sonuc));
```

## Lambda İfadeleri

**Söz Dizimi**

`[yakalama](parametreler) -> dönüş { gövde }`

| Yakalama | Anlam |
|---|---|
| `[]` | Hiçbir şey |
| `[=]` | Tümü değer ile |
| `[&]` | Tümü referans ile |
| `[x]` | Sadece x değer ile |
| `[&x]` | Sadece x referans ile |
| `[=, &x]` | Tümü değer, x referans |
| `[this]` | this işaretçisi |

**Örnekler**

```
// Basit lambda
auto kare = [](int x){ return x * x; };
kare(5); // 25

// Dis degisken yakalama
int esik = 80;
auto yuksek = [esik](int n){
    return n >= esik;
};

// Generic lambda (auto parametre)
auto topla = [](auto a, auto b){
    return a + b;
};
topla(3, 4);      // 7
topla(1.5, 2.3); // 3.8
```

**std::function**

```
#include <functional>
std::function<int(int,int)> islem;
islem = [](int a, int b){ return a+b; };
islem(3, 4); // 7
// Uyari: type erasure maliyeti var!
```

**Algoritma Seçim Rehberi**

Arama → `find` / `find_if` (sıralıysa `binary_search`)
Sıralama → `sort` (kararlı: `stable_sort`)
Dönüştürme → `transform`
Silme → C++20 `erase_if` veya remove-erase
Toplama → `accumulate`
Bölme → `partition`